Large Language Models — Final Project Presentation

LLM Hallucination Detection & Mitigation

A Retrieval-Augmented, Claim-Verified, Multi-Round Self-Correction Pipeline
evaluated on the HaluEval QA benchmark across 400 questions

Config A Closed-book | Config B Plain RAG | Config C Grounded RAG | Config D Verify + Mitigate

Team: Thrivikram (23BCS100) · Kuladeep (23BDS021) · Harshith (23BDS058)
Sadan (23BDS048) · Chaitra (23BDS016)
Course: Large Language Models, IIIT Dharwad

Generator: openai/gpt-oss-20b (Groq)
Verifier: DeBERTa-v3-base-mnli-fever-anli
Embeddings: all-MiniLM-L6-v2 (384d)

Presentation Roadmap

What we will cover, in order

1. Motivation & research questions
2. Dataset, splits and pre-registered protocol
3. System architecture (4 configurations)
4. Component deep-dives with code
5. Retrieval evaluation
6. Answer-quality results per configuration
7. Error analysis & taxonomy
8. Claim extraction + NLI verification
9. The novel Lexical Support metric
10. Threshold tuning (F0.5) & detector AUC
11. Mitigation loop (Config D)
12. Verification-only baselines (E5/E6)
13. Final test evaluation with CIs
14. Pre-set criteria verdict
15. Post-hoc audit of remaining errors
16. Cost, latency & engineering challenges
17. Limitations, future work, reproducibility

1. Motivation & Problem Definition

Why hallucination is worth engineering against — and how we measure it

- The failure mode: plausible, fluent, factually wrong answers
- Why closed-book generation is unsafe for factual QA
- What a hallucination proxy is (and what it is not)
- Our five research questions

The Problem: Confident, Plausible, Wrong

Why parametric memory alone is not a dependable knowledge source

- **LLMs answer from parametric memory** when no external evidence is supplied — the weights store statistical regularities of text, not a verified fact store.
- The failure is not gibberish: the model produces a **fluent, well-formed, confident answer** that is simply false ("Pilot" for "astronaut", "Matt Maiellaro" for "Dave Willis").
- In our closed-book baseline this happened on **65% of 200 test questions** — the model almost always answers, and is wrong most of the time.
- This matters because downstream systems cannot distinguish a fabricated answer from a correct one: there is no uncertainty signal, no citation, no provenance.

65%

of test questions were answered confidently but wrongly (closed-book)

35%

closed-book accuracy — the model answered almost every question

0

abstentions in the closed-book configuration

src/run_baseline.py - no evidence, no abstention option

```
# Config A - Closed-book baseline (src/run_baseline.py)
PROMPT = (
    "Answer the question in as few words as possible. "
    "Give only the answer, no explanation.\n\n"
    "Question: {q}\nAnswer:"
)

out = ask_llm(PROMPT.format(q=r["question"]), with_meta=True)
pred = out["text"]          # the model never says "I don't know"
```

Research Questions & Objectives

Five questions that the experiment answers with numbers

- **RQ1 — How much does retrieval help?** Does adding retrieved evidence to the prompt move accuracy, and by how much does it reduce confident errors?
- **RQ2 — Can we suppress hallucinations without paying accuracy?** Does a grounded prompt that forbids outside knowledge and permits abstention trade a little accuracy for a large drop in wrong answers?
- **RQ3 — Can we detect hallucinations automatically?** Does an NLI verifier (DeBERTa-v3) plus a lexical support score separate supported from unsupported claims (AUC)?
- **RQ4 — Is active mitigation better than passive rejection?** Does regenerating with expanded evidence beat simply abstaining whenever the detector flags an answer?
- **RQ5 — Does it generalize?** Do the detector and the loop keep working on an independent, pre-registered test split?

Contributions of this project

- A four-configuration study of hallucination in LLM QA, run end-to-end on 400 questions with a pre-registered test protocol and bootstrap confidence intervals.
- A **novel Lexical Support metric** that fixes the coreference failure mode of off-the-shelf NLI verifiers, cutting false alarms by more than half (37 -> 24 on dev).
- A **multi-round dynamic self-correction loop** (top-5, then top-10 retrieval) with a keep-best-supported fallback that recovers 19 net correct answers over 400 questions with zero regressions.
- An honest **negative result**: the pre-set primary criterion was NOT met on test, reported and analysed rather than re-tuned.

2. Dataset, Splits & Experimental Protocol

HaluEval QA, fixed splits with seed 42, and a pre-registered test plan

- HaluEval QA: 10,000 rows with knowledge, question, gold and hallucinated answer
- Fixed dev / val / test splits — 100 / 300 / 500 questions
- Pre-registered protocol: detector, thresholds, prompts frozen before test
- Reproducibility: greedy decoding, disk cache, seed 42, saved artefacts

Dataset: HaluEval QA (Li et al., EMNLP 2023)

The standard benchmark for hallucination in question answering

- Each row contains a **knowledge passage**, a **question**, a **right answer** and a **hallucinated answer** (a fluent but false answer written by annotators).
- The corpus for retrieval is built from the `knowledge` field of **all 10,000 rows** — not just the evaluated splits — so the correct passage must compete with thousands of distractors.
- Cleaning: drop empty fields, strip whitespace, drop duplicate questions -> **10,000 unique rows**.
- Each chunk keeps the ids of the rows it came from, which is what makes **Recall@K** measurable.

Field	Meaning	Used for
knowledge	Wikipedia-style evidence passage	Retrieval corpus + oracle evidence
question	Natural-language question	Query, claim rewriting
right_answer	Correct short answer	Gold label for scoring
hallucinated_answer	Fluent but false answer	Labeled pairs for detector AUC

src/make_splits.py - download, clean, shuffle once, split

```
# src/make_splits.py
URL = "https://raw.githubusercontent.com/RUCAIBox/HaluEval/main/data/qa_data.json"
df = pd.read_json(RAW, lines=True)
df = df.dropna(subset=REQUIRED)
df = df[(df[REQUIRED] != "").all(axis=1)]
df = df.drop_duplicates(subset="question")          # -> 10,000 unique rows
df = df.sample(frac=1, random_state=42).reset_index(drop=True)   # fixed seed
df.insert(0, "id", df.index)                        # permanent ids
dev  = df.iloc[:100]      # development
val  = df.iloc[100:400]   # validation (first 100 used for tuning)
test = df.iloc[400:900]   # reserved (first 200 used once, at the end)
```

Experimental Protocol (Pre-Registered)

Everything that touches the test split was frozen before the test split was opened

What was FROZEN before test

- The **detector**: mean(NLI entailment, lexical support), threshold τ = 0.55
- The **retry schedule**: top-5 passages, then top-10 passages
- **All four prompts** (baseline, plain RAG, grounded, claim rewriting)
- **Both fallback policies**: keep-best-supported (primary), abstain (secondary)
- The **label code** (`src/labels.py`) and the selection rule for the detector
- The **pre-set success criteria** for the mitigation loop

src/final_eval.py

```
# src/final_eval.py - bootstrap CIs and the criteria check
N_BOOT = 5000
rng = np.random.default_rng(42)
idx = rng.integers(0, n, size=(N_BOOT, n))
def ci(arr):
    return np.percentile(arr.astype(float)[idx].mean(axis=1), [2.5, 97.5])
def ci_diff(x, y):
    return np.percentile((x.astype(float)-y.astype(float))[idx].mean(axis=1), [2.5, 97.5])
primary = D["ver"].sum() > C["ver"].sum() and D["wrong_ver"].sum() <= C["wrong_ver"].sum()
secondary = (D["wrong_ver"].sum() < B["wrong_ver"].sum()) \
          and (D["ver"].mean() >= B["ver"].mean() - 0.03)
```

Pre-set criteria (decided in advance)

PRIMARY: Config D (keep) has MORE verified-correct answers than Config C AND no more wrong-and-answered answers than C.

SECONDARY: Config D (keep) has FEWER wrong-and-answered answers than Config B and a verified-correct rate within 3 points of B.

- **Statistics:** 95% bootstrap intervals, 5,000 resamples of the questions; paired differences between configurations on the same resampled questions.
- **Order of work:** day 1 baseline + B; day 2 C, claims, NLI, re-scoring; day 3 mitigation loop and the final evaluation. Then detector evaluation and verification-only baselines.

3. System Architecture

Four configurations, one pipeline: retrieval -> generation -> verification -> mitigation

- Config A Closed-book baseline
- Config B Plain RAG (top-3 dense retrieval)
- Config C Grounded RAG + abstention
- Config D Verification gate + multi-round mitigation loop

End-to-End Architecture

The full pipeline as implemented in src/



Top band: offline corpus construction (runs once). Middle band: query-time retrieval and generation. Bottom band: the Config D verification gate and mitigation loop — the detector only ever sees the RETRIEVED evidence, the realistic deployment setting; the row's own knowledge text is used separately, as an oracle, to separate retrieval errors from verifier errors. Everything except the LLM call runs locally on CPU (MacBook Air, 8 GB RAM). Every LLM response is cached on disk, so re-running any analysis costs zero tokens.

The Four Configurations Compared

One change at a time: evidence in, grounding in, verification in

Config	Name	What changes	Evidence	May abstain?	Status
A	Closed-book baseline	Answers from parametric memory only	none	no	dev, test
B	Plain RAG (top-3)	Retrieved passages added to the prompt	top-3	no	dev, val, test
C	Grounded RAG + abstention	Prompt restricted to evidence; must say "Insufficient evidence" otherwise	top-3	yes	dev, val, test
D	RAG + verification & mitigation	Starts from C; flagged/abstained items are re-retrieved (K=5, 10) and regenerated	top-3 -> 5 -> 10	yes	dev, val, test

The four prompts (verbatim)

A: Answer the question in as few words as possible. Give only the answer, no explanation.

B: Use the evidence below to answer the question. Give only the answer in as few words as possible, no explanation.

C: Answer the question using ONLY the evidence below. Do not use outside knowledge. If the evidence does not contain the answer, reply exactly: Insufficient evidence. Otherwise give only the answer in as few words as possible, no explanation.

D: same grounded prompt as C, but re-run with top-5 then top-10 passages when triggered.

4. Component Deep-Dives

Every module of the pipeline, with the code that implements it

- Corpus construction: chunking + dense embeddings
- Retriever: exact cosine search (and why FAISS was abandoned)
- LLM client: caching, throttling, retries
- Metrics: three scoring tiers + abstention detection
- Claim rewriting, NLI verification, detector, mitigation loop

Component 1 — Corpus Construction: Chunking & Embedding

src/build_index.py

- Word-based chunks of at most **120 words** with **20% overlap**; identical chunk texts are merged and the source row ids are remembered.
- Result: **10,040 unique chunks, ~55 words average** — nearly every row stays a single chunk.
- Embeddings: `all-MiniLM-L6-v2`, **384 dimensions, L2-normalized** (so cosine similarity = dot product).
- Embedding all 10,040 chunks took **~20 seconds on CPU**.

```
def chunk_words(text, size, overlap):
    words = text.split()
    if len(words) <= size:
        return [" ".join(words)]
    step = size - overlap
    chunks = []
    for start in range(0, len(words), step):
        chunks.append(" ".join(words[start:start + size]))
        if start + size >= len(words):
            break
    return chunks
# identical chunk texts are merged; we remember which rows they came from
text_to_ids = {}
for _, r in df.iterrows():
    for c in chunk_words(r["knowledge"], CHUNK_WORDS, OVERLAP):
        text_to_ids.setdefault(c, set()).add(int(r["id"]))
# embed: cosine similarity == dot product on normalized vectors
emb = model.encode([c["text"] for c in chunks], batch_size=64,
                   normalize_embeddings=True).astype("float32")
np.save(f"data/emb_{CHUNK_WORDS}.npy", emb)    # (10040, 384)
```

Artefact	Shape	Size
chunks_120.jsonl	10,040 chunks	~2.1 MB
emb_120.npy	(10040, 384) float32	15.4 MB
avg words / chunk	—	55

Why 120 words?

MiniLM truncates its input at ~256 word pieces (~150 words). Larger chunks would be silently cut by the encoder, so 120 words keeps every chunk fully representable.

Component 2 — Dense Retriever (and the FAISS Segfault)

src/retrieve.py

- Query and chunks are embedded with the same model; search = **exact cosine similarity** over all 10,040 vectors, computed as a single matrix product.
- FAISS crashed Python with a **segmentation fault** (FAISS 1.15.1 + PyTorch + Python 3.14 on Apple Silicon). A flat index over 10K vectors gives identical results, so we replaced it.
- Retrieval adds **~0.015 s per question** — negligible next to the LLM call.

```
class Retriever:
    def __init__(self, chunk_words=120):
        self.chunks = [json.loads(l) for l in open(f"data/chunks_{chunk_words}.jsonl")]
        self.emb = np.load(f"data/emb_{chunk_words}.npy")
        self.model = SentenceTransformer(EMB_MODEL)
        self.index = None
        if faiss is not None:                    # optional backend
            self.index = faiss.IndexFlatIP(self.emb.shape[1])
            self.index.add(self.emb)
    def search_many(self, queries, k=3):
        q = self.model.encode(queries, batch_size=64,
                              normalize_embeddings=True,
                              convert_to_numpy=True).astype("float32")
        if self.index is not None:               # FAISS path
            scores, idx = self.index.search(q, k)
        else:                                    # numpy exact cosine
            sims = q @ self.emb.T                # (n_q, 10040)
            idx = np.argsort(-sims, axis=1)[:, :k]
            scores = np.take_along_axis(sims, idx, axis=1)
        return [{**self.chunks[i], "score": float(s)} for s, i in zip(scores, idx)]
```

Engineering problem #7

Symptom: building embeddings worked, but the parallel FAISS search crashed the interpreter with a segmentation fault.

Cause: a known conflict between FAISS and PyTorch on Apple Silicon, made more likely by the very new Python 3.14 and FAISS 1.15.1.

Fix: vectorized exact cosine with NumPy (`q @ emb.T`) — same results, milliseconds, no native dependency. FAISS remains an optional switch (`USE_FAISS=1`).

Component 3 — The LLM Client: Cache, Throttle, Retry

src/llm.py — the module that made ~950K tokens of experiments possible on a free tier

- **SHA-256 disk cache** keyed by (model, reasoning, temperature, max_tokens, prompt) — no prompt is ever paid for twice, and all re-analysis is free.
- **Sliding-window rate limiter**: ≥2.2 s between calls and a 6,500 tokens/minute budget, safely under Groq's 8,000 TPM ceiling.
- **Auto-retry on reasoning starvation**: `gpt-oss-20b` sometimes spends the whole token budget thinking and returns an empty answer — the client doubles `max_tokens` (to 2048) and retries.
- Every call records **latency, model version and token usage**, which feeds the cost analysis.

Provider migration (problem #1)

The first attempt used Gemini via Google AI Studio: the free tier allows only 20 requests per day — hopeless for 100+ questions per experiment. We checked the usage dashboard, stopped the run and switched to Groq. The cache and limiter were written once and never changed afterwards.

src/llm.py

```
def _key(prompt, temperature, max_tokens):
    raw = f"{MODEL}|{REASONING}|{temperature}|{max_tokens}|{prompt}"
    return hashlib.sha256(raw.encode()).hexdigest()

def _throttle(est_tokens):
    """Sleep until both the request gap and the tokens-per-minute budget allow."""
    while True:
        now = time.time()
        while _window and now - _window[0][0] > 60:
            _window.popleft()
        used = sum(t for _, t in _window)
        gap = MIN_GAP - (now - _last_call)
        if used + est_tokens > TPM_BUDGET and _window:
            time.sleep(max(1.0, 60 - (now - _window[0][0]) + 0.5))
        elif gap > 0:
            time.sleep(gap)
        else:
            return

# reasoning model returned nothing: double the budget and retry
if not text and choice.get("finish_reason") == "length" and cur_max < 2048:
    cur_max *= 2
    continue

# HTTP 429 -> honour Retry-After; daily limit -> stop with a clear message
if r.status_code == 429:
    delay = float(r.headers.get("retry-after", 20))
    time.sleep(delay + 1)
```

Component 4 — Multi-Tier Evaluation Metrics

src/metrics.py — because the strict metric was undercounting correct answers

Three tiers of answer correctness

Tier	Definition	Catches
Strict "contains"	normalized gold string appears in the normalized prediction	the original metric, kept unchanged for traceability
Lenient "contains"	strict match OR every key word of the gold appears in the prediction	Unicode dashes, number words, extra middle names
Manually verified	lenient match, or a short-form answer that was not judged incomplete	synonyms, nicknames, stage names

- **Abstained** — 8 abstention phrases are recognised ("insufficient evidence", "unknown", "not mentioned", ...).
- **Wrong & answered** — the model answered and it is not correct. This is our **hallucination proxy**.
- **Evidence had answer** — does the gold string literally appear in the retrieved evidence? A ceiling on what a faithful answer can achieve (unreliable for yes/no questions).
- **Exact Match, token F1** — standard, unchanged.

src/metrics.py

```
def lenient_tokens(s):
    s = unicodedata.normalize("NFKC", str(s)).lower().translate(_TRANS)
    s = "".join(ch for ch in s if ch not in string.punctuation)
    return [NUM_WORDS.get(t, t) for t in s.split() if t not in _STOP]

def lenient_match(pred, gold):
    """Strict contains, OR every key word of the gold answer is in the prediction."""
    if contains_match(pred, gold):
        return 1.0
    g, p = lenient_tokens(gold), lenient_tokens(pred)
    return float(len(g) > 0 and set(g) <= set(p))

def short_form_match(pred, gold):
    """Every word of the prediction appears in the gold answer."""
    g, p = lenient_tokens(gold), lenient_tokens(pred)
    return float(len(p) > 0 and set(p) <= set(g))

def is_abstain(pred):
    p = normalize(pred)
    return any(s in p for s in ["insufficient evidence", "unknown",
            "not enough information", "i do not know", "cannot be determined",
            "not mentioned"])
```

Why Three Tiers? A Concrete Audit of the Strict Metric

Manual review of the cases the strict metric marked wrong despite the evidence being present

Question (shortened)	Gold answer	Model answer	Verdict
Azfar Hussain ... official language of which country?	People's Republic of Bangladesh	Bangladesh	Correct (short form)
Miss A member in While You Were Sleeping	Bae Suzy	Suzy	Correct (stage name)
Darrelle Revis's team two years after he was drafted	The New York Jets	Jets	Correct
When was the 2007 movie released?	January 12, 2007	2007	Incomplete -> not counted
What job do both Selpin and Cavani have?	film director and screenwriter	Film director	Incomplete -> not counted
Which UK equestrian weekly is Mark Phillips a columnist in?	Horse & Hound	The Horse	Wrong (different magazine)

11 -> 2

Config C answers called 'wrong despite evidence': 11 under strict, only 2 truly wrong

12 of 18

Config B 'wrong despite evidence' cases were scoring artifacts, not model errors

3 tiers

strict / lenient / manually verified — all three are reported for every configuration

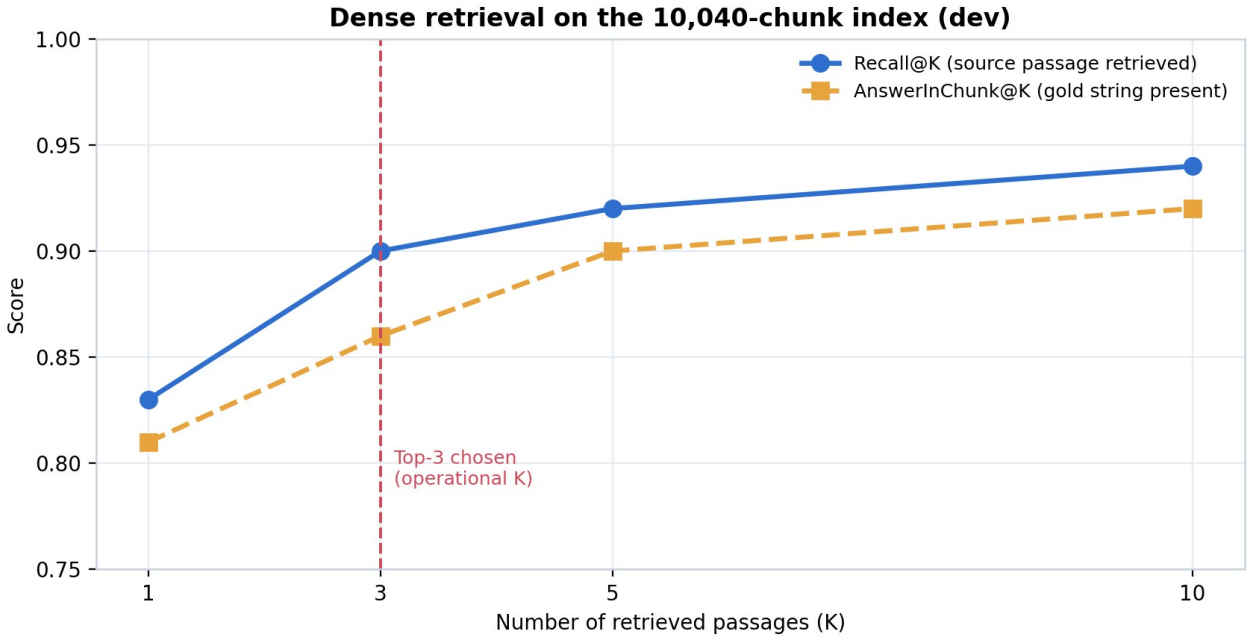
5. Retrieval Evaluation

How good is the evidence the generator sees?

- Recall@K and AnswerInChunk@K on a 10,040-chunk index
- MRR@10 = 0.871 — the correct passage is ranked first 83% of the time
- Top-3 chosen as the operating point
- Failure analysis: 10 dev questions where retrieval missed

Retrieval Performance (10,040-chunk dense index, dev)

src/eval_retrieval.py



Metric	@1	@3	@5	@10	MRR@10
Recall@K (source passage in top-K)	0.830	0.900	0.920	0.940	0.871
AnswerInChunk@K (gold string in top-K)	0.810	0.860	0.900	0.920	—

- The correct passage is **ranked first for 83% of questions** and inside the top 3 for **90%** — hence top-3 as the operating point for Configs B and C.
- **Retrieval is not the bottleneck of the detector**: with oracle evidence the pair AUC improves only from 0.73 to 0.79, so most of the verifier's errors are verifier errors, not retrieval errors.
- **Caveat**: the corpus is the dataset's own `knowledge` field, so retrieval is *easier* than open-domain search. These numbers should be read in that light.

src/eval_retrieval.py - Recall@K, AnswerInChunk@K, MRR

```
def evaluate_recall(df, results, KS=[1, 3, 5, 10]):
    hit, ans_hit, rr = {k: [] for k in KS}, {k: [] for k in KS}, []
    for (_, row), res in zip(df.iterrows(), results):
        qid = int(row["id"])
        gold = normalize(row["right_answer"])
        rel = [qid in c["source_ids"] for c in res]
        has_ans = [gold != "" and gold in normalize(c["text"]) for c in res]
        first = next((i for i, x in enumerate(rel) if x), None)
        rr.append(1 / (first + 1) if first is not None else 0.0)
        for k in KS:
            hit[k].append(any(rel[:k]))
            ans_hit[k].append(any(has_ans[:k]))
    return hit, ans_hit, rr
```

Why Answers Fail: An Error Taxonomy

src/error_analysis.py — every question sorted into an outcome category per configuration

Outcome category	Config B (plain RAG)	Config C (grounded RAG)	Meaning
correct	83	79	answer matches the gold string
abstain_appropriate	—	10	abstained although the evidence lacked the answer
abstain_over_cautious	—	6	abstained although the evidence did contain it
wrong_despite_evidence	3	2	evidence was there but was misread
wrong_evidence_lacks_answer	12	—	retrieval failed and the model guessed

Where Config B's 15 remaining wrong answers came from (manual audit, dev)

Cause	Count	Example
Retrieval failed and the model guessed (incl. "No such player.")	8	the answer simply is not in the retrieved evidence
Evidence present but misread	3	"¡Viva Zapata!" for an album question; "Yes." to a statement-style question
Yes/no question answered incorrectly	1	—
Incomplete answer (year instead of full date; one of two occupations)	2	"2007" vs "January 12, 2007"
Malformed question (not counted as correct)	1	a broken question in the benchmark

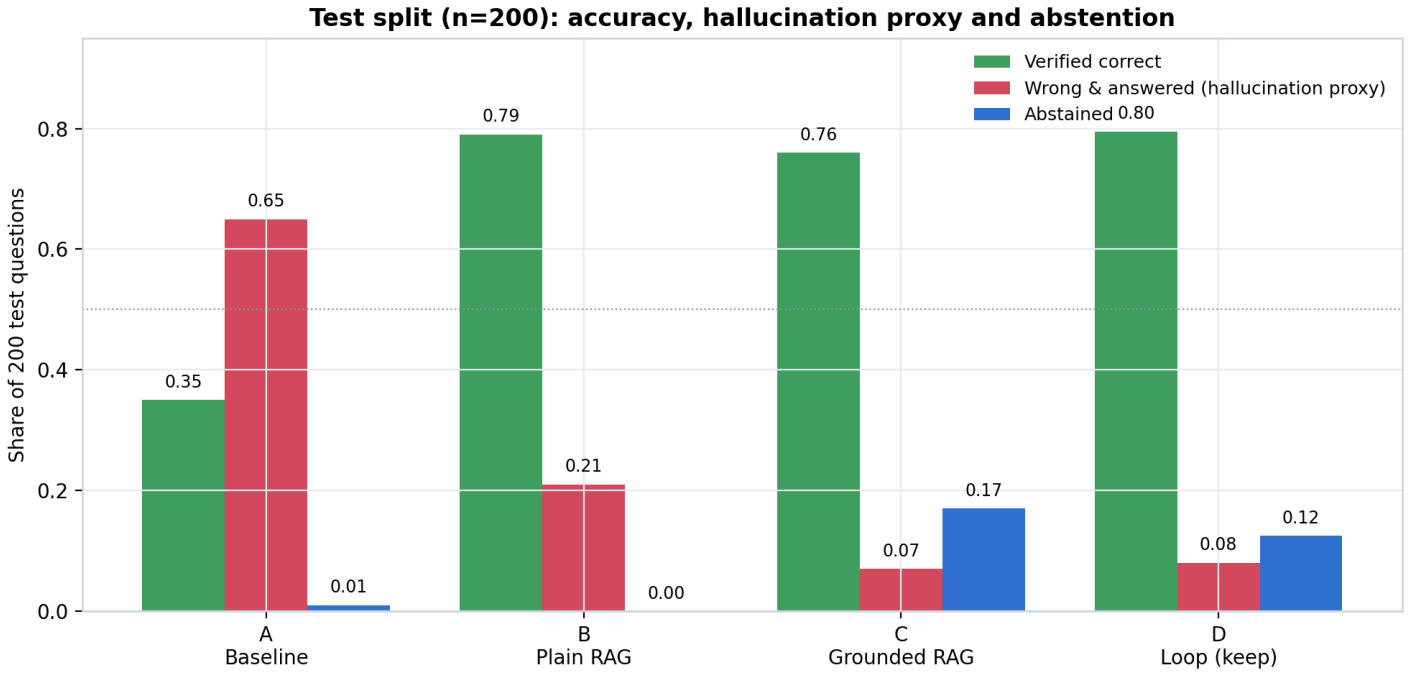
6. Results: Answer Quality Across Configurations

The core experimental results, on dev, validation and test

- Config A: 35% correct, 65% confidently wrong
- Config B: 79% correct but 21% confidently wrong
- Config C: 76% correct, only 7% wrong (17% abstain)
- Config D: 80% correct, 8% wrong, 12% abstain

Headline Result — 200 Test Questions

Config A -> B -> C -> D, verified labels



- **Retrieval is the single biggest lever**: +43.5 points of verified accuracy over the closed-book baseline (95% CI [+36.0, +51.5]).
- **Grounding removes errors without a real accuracy cost**: wrong-and-answered falls from 21% to 7% (Δ = −14.5, CI [−20.0, −9.5]) while accuracy drops only 3 points (not significant).
- **The loop recovers abstentions**: +4.0 points of accuracy over C (CI [+1.5, +7.0]) with **zero regressions** on the 151 answers C got right.

The honest caveat

The loop produced 16 wrong answers on test versus C's 14. The pre-set primary criterion required **no** increase in errors, so it was **NOT MET**. We did not re-tune after seeing this.

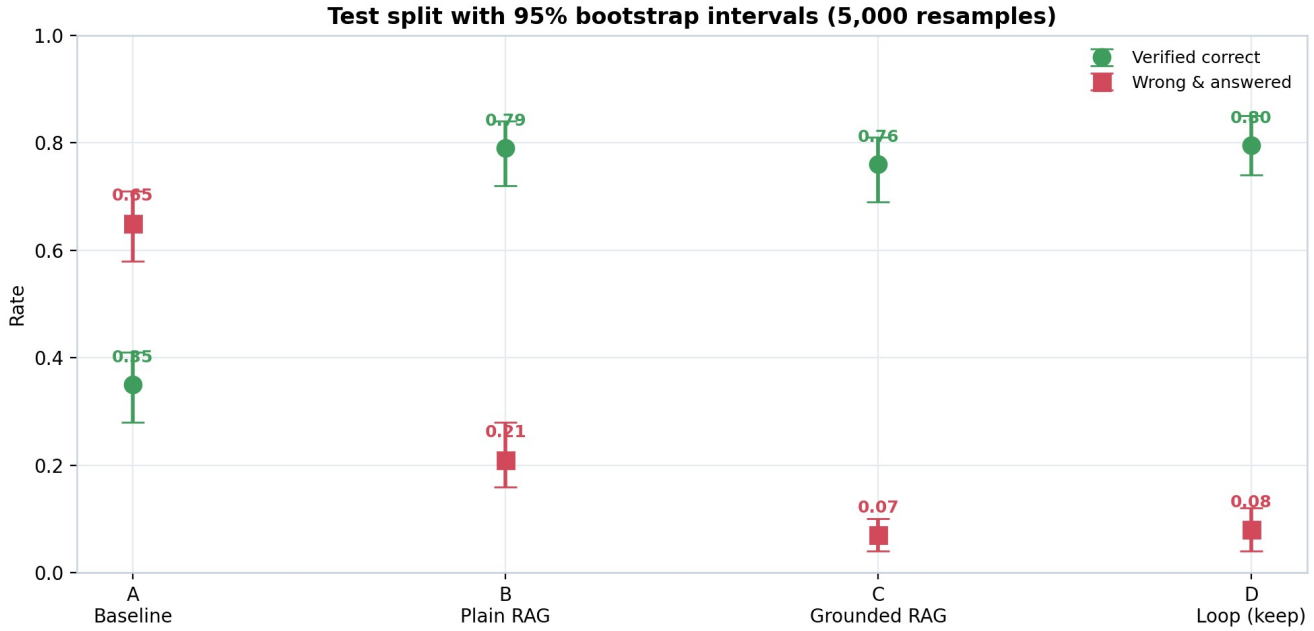
Full Answer-Quality Results Across All Three Splits

400 evaluated questions (dev 100, val 100 used, test 200)

Split	Configuration	Strict	Lenient	Verified	Abstained	Wrong & ans. (lenient)	Wrong & ans. (verified)
dev	A. Baseline	0.31	0.31	0.33	0.01	0.68	0.66
(100 Q)	B. Plain RAG (Top-3)	0.71	0.77	0.83	0.02	0.21	0.15
	C. Grounded RAG (Top-3)	0.72	0.76	0.79	0.17	0.07	0.04
	D. Loop (Keep Best)	0.78	0.82	0.85	0.11	0.07	0.04
	D. Loop (Abstain if Flagged)	**0.71**	**0.75**	**0.78**	**0.20**	**0.05**	**0.02**
val	B. Plain RAG (Top-3)	0.79	0.81	0.87	0.01	0.18	0.12
(100 Q)	C. Grounded RAG (Top-3)	0.75	0.77	0.83	0.12	0.11	0.05
	D. Loop (Keep Best)	0.80	0.82	0.88	0.09	0.09	0.03
	D. Loop (Abstain if Flagged)	0.76	0.78	0.83	0.14	0.08	0.03
test	A. Baseline	0.30	0.31	0.35	0.01	0.69	0.65
(200 Q)	B. Plain RAG (Top-3)	0.71	0.73	0.79	0.00	0.27	0.21
	C. Grounded RAG (Top-3)	0.69	0.71	0.76	0.17	0.12	0.07
	D. Loop (Keep Best)	0.71	0.74	0.80	0.12	0.13	0.08
	D. Loop (Abstain if Flagged)	**0.68**	**0.71**	**0.76**	**0.17**	**0.12**	**0.07**

Statistical Rigor: Paired Differences with Bootstrap CIs

Test split, 200 questions, 5,000 bootstrap resamples, verified labels



Paired difference	Δ verified correct [95% CI]	Δ wrong & answered [95% CI]
D keep − C grounded	+0.040 [+0.015, +0.070]	+0.010 [0.000, +0.025]
D keep − B plain RAG	+0.010 [−0.035, +0.055]	−0.135 [−0.190, −0.085]
C grounded − B plain RAG	−0.030 [−0.080, +0.020]	−0.145 [−0.200, −0.095]
B plain RAG − A baseline	+0.435 [+0.360, +0.515]	−0.430 [−0.510, −0.355]
D keep − A baseline	+0.445 [+0.370, +0.520]	−0.565 [−0.635, −0.495]

- **Retrieval effect is huge and unambiguous**: +43.5 points of accuracy, CI far from zero.
- **Grounding's accuracy cost is not significant** (−3.0, CI [−8.0, +2.0]) while its error reduction is highly significant (−14.5, CI [−20.0, −9.5]).
- **D vs B**: statistically indistinguishable accuracy (CI contains 0) but **13.5 points fewer wrong answers** (CI entirely below 0).

Quality vs. Cost: Lexical Overlap, Latency and Tokens

Retrieval adds milliseconds; grounding costs almost nothing; the loop costs ~1.8x

Config	Split	Exact Match	Token F1	Avg LLM latency	Avg retrieval latency	Avg tokens/Q
A. Baseline	dev	0.250	0.301	0.73 s	—	290
B. Plain RAG (Top-3)	dev	0.660	0.731	0.77 s	~0.015 s	433
C. Grounded RAG (Top-3)	dev	0.630	0.717	0.69 s	~0.015 s	434
A. Baseline	test	0.260	0.355	0.98 s	—	311
B. Plain RAG (Top-3)	test	0.630	0.739	0.87 s	~0.015 s	446
C. Grounded RAG (Top-3)	test	0.615	0.702	0.79 s	~0.015 s	457
D. Mitigation Loop	test	—	—	~3.8 s (when triggered)	~0.030 s	810 (1.8x C)

- **RAG costs ~50% more tokens** (290 -> 433) but the API latency is unchanged — the extra tokens are input context, not extra generation.
- **Grounding is free**: Config C uses the same tokens and is actually marginally faster than B.
- **The loop roughly doubles the cost** (810 tokens/query, ~3.8 s when triggered) and triggers on only 19–27% of questions — so the **average** cost increase is much smaller than the worst case.

7. Detection: Claim Verification & the Detector

Turning a short answer into a checkable claim, and scoring whether the evidence supports it

- Atomic claim rewriting via LLM
- DeBERTa-v3 NLI verification (entail / neutral / contra)
- The Lexical Support metric — our novel contribution
- Threshold selection by F0.5 on validation

Component 5 — Atomic Claim Extraction

src/make_claims.py — short answers cannot be checked, claims can

- Answers are one to three words long, so sentence splitting would achieve nothing. Instead the LLM **rewrites each question+answer pair as one self-contained declarative sentence**.
- Applied to **four** answer types so the detector cannot exploit stylistic differences: HaluEval's `right_answer` and `hallucinated_answer` (both rewritten), and the non-abstained answers of Configs B and C.
- Scale: **381 claims** on dev (98 from B, 83 from C), **387** on validation, **765** on test.

Source	Claim count (dev)	Role
right	100	supported (label 0)
halluc	100	hallucinated (label 1)
B	98	pipeline answer
C	83	pipeline answer

src/make_claims.py

```
PROMPT = (
    "Rewrite the question and its answer as ONE short, self-contained declarative sentence "
    "that states the answer as a fact. Do not add any information that is not in the "
    "question or the answer. If the answer is Yes or No, state the fact accordingly. "
    "Output only the sentence.\n\n"
    "Question: {q}\nAnswer: {a}\nSentence:"
)

items = []                      # four answer types, all rewritten identically
for _, r in df.iterrows():
    qid = int(r["id"])
    if "right"  in SOURCES: items.append((qid, "right",  r["question"], str(r["right_answer"])))
    if "halluc" in SOURCES: items.append((qid, "halluc", r["question"], str(r["hallucinated_answer"])))
    for src, preds in (("B", B), ("C", C)):
        a = preds.get(qid, "")
        if src in SOURCES and a and not is_abstain(a):
            items.append((qid, src, r["question"], a))

for qid, src, q, a in tqdm(items):
    out = ask_llm(PROMPT.format(q=q, a=a), with_meta=True)
    rows.append({"id": qid, "source": src, "claim": out["text"]})
```

Worked example

Q: Who won the singles title at a 1973 tournament held at a Los Angeles arena designed by Charles Luckman?

A: Margaret Court

Claim: Margaret Court won the singles title at a 1973 tournament held at a Los Angeles arena designed by Charles Luckman.

Component 6 — NLI Verification Engine

src/nli.py / src/verify.py — entailment between the evidence and the claim

src/nli.py

```
NLI_MODEL = os.getenv("NLI_MODEL", "MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli")
_tok   = AutoTokenizer.from_pretrained(NLI_MODEL)
_model = AutoModelForSequenceClassification.from_pretrained(NLI_MODEL).eval()

def _find(prefix):
    for i, label in _model.config.id2label.items():
        if str(label).lower().startswith(prefix):
            return int(i)
    raise ValueError(f"No '{prefix}' label in {_model.config.id2label}")

E, N, C = _find("entail"), _find("neutral"), _find("contra")

@torch.no_grad()
def nli_scores(premises, hypotheses, batch_size=8):
    out = []
    for i in tqdm(range(0, len(premises), batch_size)):
        enc = _tok(premises[i:i+batch_size], hypotheses[i:i+batch_size],
                   truncation="only_first", max_length=512,
                   padding=True, return_tensors="pt")
        probs = torch.softmax(_model(**enc).logits, dim=-1).numpy()
        out.extend({"entail": float(r[E]), "neutral": float(r[N]),
                    "contra": float(r[C])} for r in probs)
    return out
```

- **Premise** = evidence (retrieved top-3 in production, the row's own knowledge for oracle analysis); **hypothesis** = the atomic claim.
- **Hallucination score = 1 − P(entailment)**.
- Two evidence sources are scored: **retrieved** (what the deployed detector sees) and **oracle** (an upper bound that separates retrieval errors from verifier errors).
- Runs on **CPU** — 381 dev claims took 328 s retrieved / 161 s oracle.

Mean P(entailment) by claim source — the signal is real

On dev: right-answer claims 0.635 (retrieved) / 0.747 (oracle) vs. hallucinated-answer claims 0.277 / 0.282. On test: 0.654 / 0.740 vs. 0.336 / 0.321. The verifier separates the two populations by roughly 0.3 in probability mass.

The Missing Entity Phenomenon — and the Lexical Support Metric

Our novel contribution: a lexical signal that repairs the NLI coreference failure

The problem

HaluEval passages often omit the entity name or start mid-sentence with a pronoun:

"It was founded in 1996 by Nobel laureate Milton Friedman" → claim about "EdChoice"

"its long-time leader Roberto Formigoni" → claim about "Network Italy"

The claim names the entity, the passage does not — **coreference breaks and NLI returns neutral**.

src/detector.py - the Lexical Support metric

```
STOP = set("""is are was were be been being has have had do does did to in on at by for with as
that which who whom whose what when where it its this these those his her their he she they
them from into than then both also not no yes more most""".split())

def lex_support(claim, question, evidence):
    """Share of the claim's NOVEL content words that appear in the evidence."""
    q = set(lenient_tokens(question))
    novel = [t for t in lenient_tokens(claim) if t not in q and t not in STOP]
    if not novel:
        return np.nan          # claim adds no new words: rely on NLI
    ev = set(lenient_tokens(evidence))
    return sum(t in ev for t in novel) / len(novel)

# combine
v["sup_mean"] = (v["entail_ret"] + lex) / 2      # deployed variant
v["sup_max"]  = np.maximum(v["entail_ret"], lex)
```

- **Raw DeBERTa NLI produced 37 false alarms** on 100 right dev claims — 6 of the 8 worst were the missing-entity case, 1 copied question details, 1 needed a comparison.
- **The fix**: compute the share of the claim's *novel* content words (not already in the question, excluding stopwords and negation) that literally appear in the evidence.
- If the claim adds no new words, fall back to the NLI entailment value.
- **Result: false alarms drop from 37 to 24 (mean) or 16 (max)** — a >50% reduction.

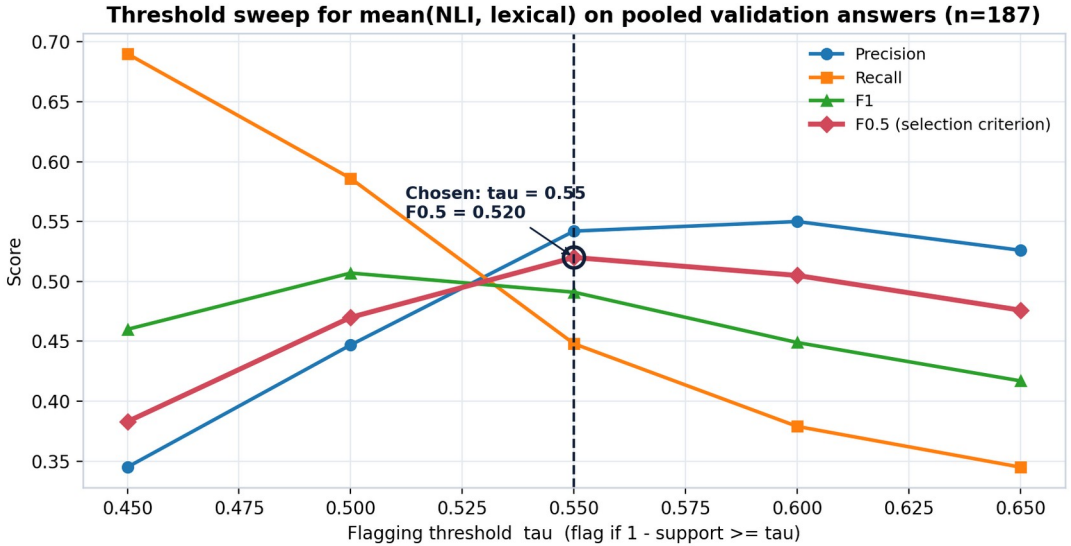
Detector variant (dev, 200 pairs)	AUC retrieved	AUC oracle	Right claims flagged (of 100)
NLI only (1 − entail)	0.732	0.792	37
Lexical support only	0.724	0.755	—
max(NLI, lexical)	0.759	0.807	16
mean(NLI, lexical)	**0.771**	**0.830**	**24**

Detector Selection: A Pre-Registered Tuning Protocol

src/tune_threshold.py — the rule was fixed before validation was opened

The rule, fixed in advance

- Choose the variant (**mean** or **max**) and the threshold (**0.30 → 0.80 in steps of 0.05**) that maximise **F0.5** on the pooled Config B + C validation answers.
- **Why F0.5?** A false alarm costs a regeneration and risks replacing a correct answer, so precision must outweigh recall.
- **Why the pipeline answers, not the HaluEval pairs?** The pairs are 50% hallucinated whereas only ~15% of pipeline answers are wrong — a threshold tuned on the pairs would flag far too many correct answers in practice. Pairs are reported as a secondary check.
- **Ties** broken by higher F1, then by the higher threshold. NLI-only is reported for reference but is not eligible.



Threshold	Flagged	True	False alarms	Precision	Recall	F1	F0.5
0.45	58	20	38	0.345	0.690	0.460	0.383
0.50	38	17	21	0.447	0.586	0.507	0.470
0.55	**24**	**13**	**11**	**0.542**	**0.448**	**0.491**	**0.520**
0.60	20	11	9	0.550	0.379	0.449	0.505
0.65	19	10	9	0.526	0.345	0.417	0.476

Selected detector (frozen for the rest of the project)

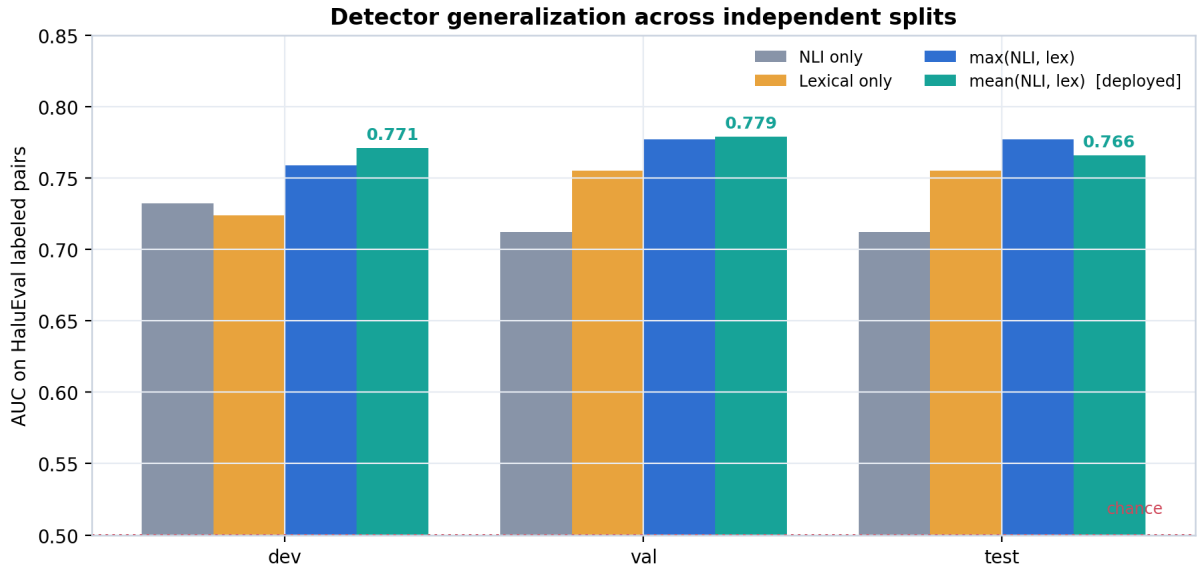
variant = mean(NLI, lexical) · evidence = retrieved · τ = 0.55 · F0.5 = 0.520

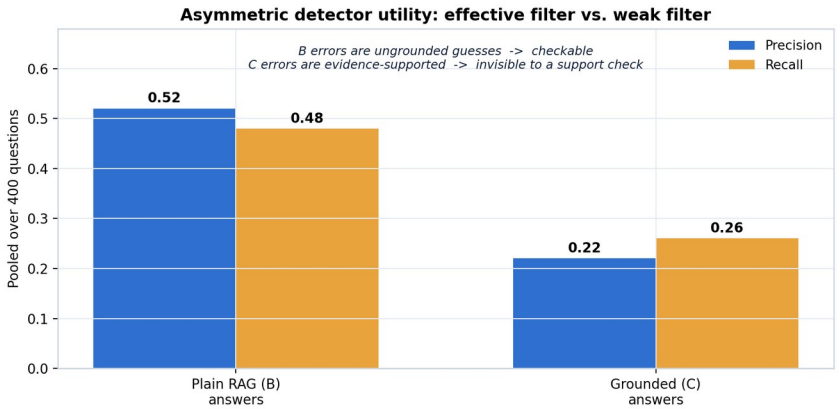
At τ = 0.55 the detector flags 24 of 187 answers (12.8%): 13 of 29 wrong answers caught (recall 0.45), 11 of 158 correct answers flagged (7.0%). Precision 0.54 ≈ **3.5x the 15.5% base rate**.

Saved to `results/chosen_detector.json` and used unchanged on dev and test.

Robustness: Label Sensitivity and Cross-Split Generalization

Did the chosen detector survive a label audit and an independent split?





Sensitivity check (validation)

Labels (mean, τ=0.55)	Wrong	AUC	Flagged (true / false)	P	R	F0.5
Original (lenient)	29	0.752	24 (13 / 11)	0.542	0.448	0.520
Corrected	17	0.832	24 (11 / 13)	0.458	0.647	0.487

- 12 of the 29 'wrong' validation answers were **short-form answers that are actually correct**; with corrected labels the best threshold of the mean variant is **still 0.55** → keep it.
- Pair-claim AUC is **stable across splits**: 0.771 (dev) → 0.779 (val) → 0.766 (test). The **classifier** generalizes.
- But the **filter utility is asymmetric**: pooled precision/recall is 0.52 / 0.48 on plain RAG answers and only 0.22 / 0.26 on grounded answers.

Why the asymmetry? Plain RAG sometimes **guesses** when retrieval fails — an evidence check can reveal that. Grounded errors are mostly **supported by the evidence** (synonyms, aliases, incompleteness), so a support check is structurally blind to them. Use the detector as a **routing signal**, never as a gate.

8. Mitigation: Config D

Multi-round dynamic self-correction with a keep-best-supported fallback

- Trigger: abstained OR flagged by the frozen detector
- Round 1: top-5 passages -> regenerate -> verify
- Round 2: top-10 passages -> regenerate -> verify
- Fallback policies: keep best-supported (primary) vs. abstain

Component 7 — The Mitigation Loop (Config D)

src/mitigate.py — the loop, in full

src/mitigate.py - Config D

```
KS = [5, 10]                        # retry depths (frozen)
T, VARIANT = chosen["threshold"], chosen["variant"]
# ---- 1. trigger: abstained, or flagged by the frozen detector ----
flag1 = (s1 is not None) and (not np.isnan(s1)) and ((1 - s1) >= T)
states.append({"id": int(qid), ..., "status": "pending" if (abst1 or flag1) else "ok",
               "cands": [] if abst1 else [(a1, s1)], "rounds": []})
# ---- 2. retry rounds: retrieve more evidence, regenerate, verify ----
for K in KS:
    pend = [s for s in states if s["status"] == "pending"]
    hits = ret.search_many([s["question"] for s in pend], k=K)
    for s, h in zip(pend, hits):
        evid_marked = "\n".join(f"[{i}] {x['text']}" for i, x in enumerate(h, 1))
        out = ask_llm(PROMPT_C.format(evidence=evid_marked, q=s["question"]), with_meta=True)
        a2 = out["text"]
        rec = {"K": K, "answer": a2, "abstained": bool(is_abstain(a2))}
        s["rounds"].append(rec)
        if rec["abstained"]:
            continue                # no answer -> try the next round
        cl = ask_llm(CLAIM_PROMPT.format(q=s["question"], a=a2), with_meta=True)
        rec["claim"] = cl["text"]
        todo.append((s, rec, evid_plain))
    sc = nli_scores([e for _, _, e in todo], [r_["claim"] for _, r_, _ in todo])
    for (s, rec, evid_plain), sco in zip(todo, sc):
        lex  = lex_support(rec["claim"], s["question"], evid_plain)
        sup2 = float(support(sco["entail"], lex))
        rec["flagged"] = bool((1 - sup2) >= T)
        s["cands"].append((rec["answer"], sup2))
        if not rec["flagged"]:                  # ACCEPT
            s["status"], s["accepted_K"], s["accepted_answer"] = "accepted", K, rec["answer"]
# ---- 3. fallback policies ----
if s["status"] == "ok":         s["final_keep"] = s["final_abs"] = s["a1"]
elif s["status"] == "accepted": s["final_keep"] = s["final_abs"] = s["accepted_answer"]
else:
    s["final_abs"]  = ABSTAIN
    s["final_keep"] = max(s["cands"], key=lambda x: (x[1], x[0] == s["a1"]))[0] if s["cands"] else ABSTAIN
```

Design rationale: about half of Config B's errors were retrieval failures (fixable by more evidence) and the detector's precision is only ~0.5 (so discarding flagged answers is unsafe) — therefore the loop never discards a flagged answer without first trying to improve it.

Mitigation Results — Dev and Validation

The loop recovers abstentions without damaging correct answers

Config	Split	Correct (lenient)	Correct (verified)	Abstained	Wrong & ans. (lenient / verified)
B plain RAG	dev	0.77	0.82	0.02	0.21 / 0.16
C grounded (start)	dev	0.76	0.79	0.17	0.07 / 0.04
D keep best-supported	**dev**	**0.82**	**0.85**	**0.11**	**0.07 / 0.04**
D abstain if flagged	dev	0.75	0.78	0.20	0.05 / 0.02
B plain RAG	val	0.81	0.87	0.01	0.18 / 0.12
C grounded (start)	val	0.77	0.83	0.12	0.11 / 0.05
D keep best-supported	**val**	**0.82**	**0.88**	**0.09**	**0.09 / 0.03**
D abstain if flagged	val	0.78	0.83	0.14	0.08 / 0.03

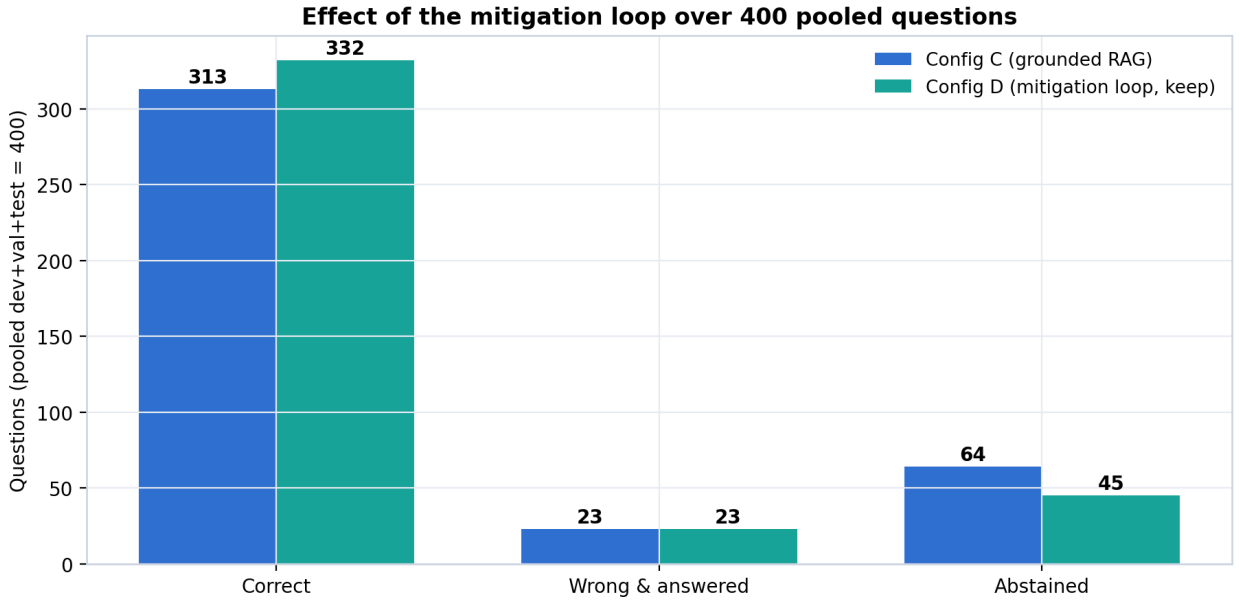
Outcomes by trigger type (dev+val pooled)

Group	Q	Outcome
C abstained	29	6 retries accepted (all correct); 3 flagged candidates kept (2 correct); 20 still abstained
C flagged	17	6 retries accepted (2 stayed correct, 3 fixed, 1 still wrong); 11 kept
Not triggered	154	150 correct, 4 wrong (2.6%)

- **Net gain over C**: verified-correct rises from 162 to 173 per 200 questions (+11); **0 of 162 correct answers were made worse**.
- **The keep policy beats the abstain policy**: pooled, keep gives 173 correct / 7 wrong, abstain gives 161 correct / 5 wrong — 12 correct answers given up to remove 2 wrong ones.
- **Both rounds are useful**: with only round 1 the loop would have given 168 correct, 8 wrong, 24 abstentions instead of 173 / 7 / 20.
- **Recovered examples**: "Insufficient evidence" → "15th" (Mincome PM), "Loud" (Rihanna album), "model" (occupation); "2007" → "January 12, 2007".

The Measured Effect of the Loop, Pooled Over 400 Questions

dev + val + test: what actually improved, and what did not



Config C → Config D (keep), verified labels	Dev	Val	Test	Pooled (400)
Correct	79 → 85	83 → 88	151 → 159	313 → 332 (+19)
Wrong and answered	4 → 4	5 → 3	14 → 16	23 → 23 (no change)
Abstained	17 → 11	12 → 9	35 → 25	64 → 45

- Of the **64 abstentions** of Config C: **16 became correct** (dev 5, val 3, test 8), 3 became wrong, 45 stayed abstained.
- Of the **23 wrong answers** of Config C: **3 were fixed**, but **3 new wrong answers appeared** — net zero change in errors.
- **All 151 correct answers of Config C survived the loop untouched.**
- **Conclusion: the loop's measured benefit is the recovery of abstentions, not the reduction of wrong answers.** The error reduction that validation suggested did **not** replicate on test.

Why did the two wrong answers get in?

"Matt Maiellaro" (support 0.50) and "Las Vegas" (support 0.57) both cleared the $\tau = 0.55$ gate — they were accepted, but barely. A stricter acceptance rule is the obvious future fix.

Mitigation on the Test Split — Detailed Breakdown

200 questions; the loop triggered for 45 of them

Group	Questions	Outcome (keep policy, verified labels)
Config C abstained	35	8 accepted retries (6 correct, 2 wrong); 2 flagged candidates kept (both correct); 25 stayed abstained
Config C answer flagged	10 (9 correct, 1 wrong)	3 accepted retries (all stayed correct); 7 fell back and were kept (6 correct, 1 wrong)
Not triggered	155	142 correct, 13 wrong

45 / 200
questions triggered
(35 abstained + 10 flagged)

11
retries accepted
(10 in round 1, 1 in round 2)

0
correct answers damaged
(all 151 stayed correct)

6%
flag rate on C's answers
(12% dev, 8% val)

- **Abstention recovery works**: 8 of 35 abstentions became usable answers (6 correct accepted retries + 2 kept candidates).
- **No error was fixed** on test (C's 14 wrong answers all survived), and **2 new errors were introduced**.
- **Round 2 earns its place**: with a single round the loop would have produced 157 correct / 16 wrong / 27 abstained instead of 159 / 16 / 25.
- **The abstain policy gave no benefit** on test: 151 correct, 15 wrong, 34 abstained — the same accuracy as Config C.
- **Cost**: 107 API calls, 70,981 tokens, 777 s (rate-limited) for the whole test run.

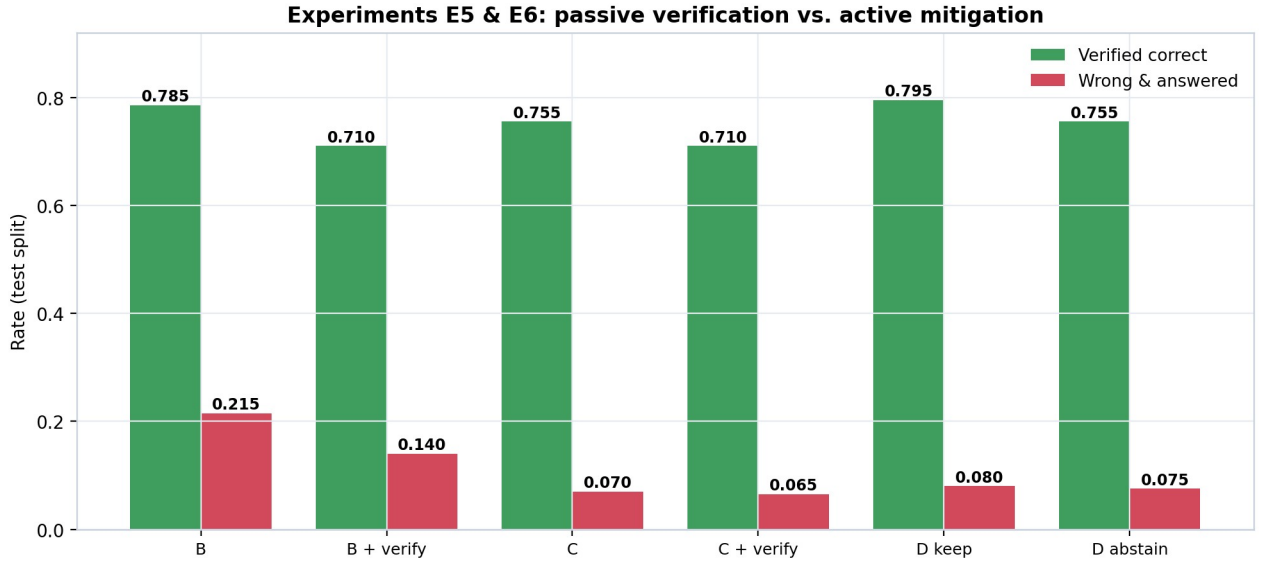
9. Verification-Only Baselines (E5 & E6)

Is it the verification, or is it the active mitigation?

- E5: plain RAG vs. plain RAG + verify-and-abstain
- E6: verification-only vs. the full mitigation loop
- Passive rejection destroys as much accuracy as it removes error
- Active mitigation dominates: +8.5 points of accuracy ($p < 0.001$)

E5 & E6: Active Mitigation Beats Passive Rejection

src/verify_only.py — computed offline from the saved claim scores, same frozen detector



Config	Test correct	Test wrong	Test abstained	Pooled correct	Pooled wrong
B. Plain RAG	0.785	0.215	0.000	0.815	0.177
B + Verify (passive abstain)	0.710	0.140	0.150	0.735	0.092
C. Grounded RAG	0.755	0.070	0.175	0.782	0.058
C + Verify (passive abstain)	0.710	0.065	0.225	0.730	0.043
D. Mitigation Loop (keep)	**0.795**	**0.080**	**0.125**	**0.830**	**0.058**
D. Mitigation Loop (abstain)	0.755	0.075	0.170	0.780	0.050

- **E5 — verification alone on plain RAG trades correct answers for wrong ones one-for-one**: −7.5 points of accuracy to remove 7.5 points of error.
- **On the grounded configuration it is strictly worse**: 4.5 points of accuracy sacrificed to remove only 0.5 points of error.
- **E6 — the loop beats verification-only by 8.5 points of accuracy** (95% CI [+5.0, +12.5]) and has a wrong-answer rate 6 points below "B + verify".
- **Config C alone (0.755 correct, 0.070 wrong) beats "B + verify" (0.710, 0.140)** — simply grounding the prompt is a better way to reduce hallucinations than detecting and abstaining afterwards.

Takeaway

Verification is only useful when it triggers an **action** better than silence. Detecting an unsupported answer is worth little if the only remedy is to throw the answer away.

10. Final Evaluation & Verdict

Pre-registered criteria, confidence intervals, and an honest negative result

- Final test table with 95% bootstrap intervals
- Primary criterion: NOT MET (16 wrong vs C's 14)
- Secondary criterion: MET (62% fewer wrong answers than plain RAG)
- Post-hoc audit of the 16 remaining wrong answers

Final Test Evaluation — 200 Questions

results/report.md §5.11.1 — verified labels with 95% bootstrap intervals

Config	Strict	Lenient	Verified correct [95% CI]	Abstained	Wrong & answered (lenient)	Wrong & answered (verified) [95% CI]
A. Baseline	0.30	0.31	0.35 [0.28, 0.41]	0.01	0.69	0.65 [0.58, 0.71]
B. Plain RAG	0.71	0.73	0.79 [0.72, 0.84]	0.00	0.27	0.21 [0.16, 0.28]
C. Grounded RAG	0.69	0.71	0.76 [0.69, 0.81]	0.17	0.12	0.07 [0.04, 0.10]
D. Loop (keep)	**0.71**	**0.74**	**0.80 [0.74, 0.85]**	**0.12**	**0.13**	**0.08 [0.04, 0.12]**
D. Loop (abstain)	0.68	0.71	0.76 [0.69, 0.81]	0.17	0.12	0.07 [0.04, 0.12]

The pre-set verdict

PRIMARY CRITERION — NOT MET

D (keep) has more verified-correct answers than C (159 vs 151) **but also more wrong-and-answered answers (16 vs 14)**, whereas the criterion allowed no increase in errors.

Two erroneous accepted retries ("Matt Maiellaro", "Las Vegas") caused the increase.

SECONDARY CRITERION — MET

D (keep) has far fewer wrong-and-answered answers than B (16 vs 43 → **62% fewer errors**) and its verified-correct rate is within 3 points of B's (0.795 vs 0.785).

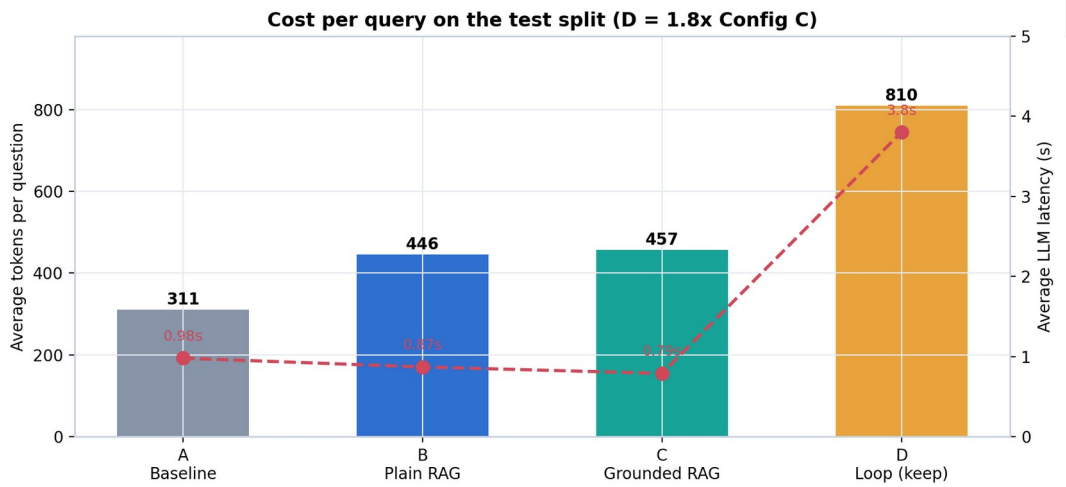
No setting was changed after the test results were seen.

Which Configuration Should You Actually Deploy?

The decision depends on which error you care about

If you want...	Deploy	Why
The lowest wrong-answer rate	Config C (grounded RAG)	0.070 wrong and 17% abstentions — the safest answer, at a 4-point accuracy cost vs. B
Maximum accuracy with few errors	Config D (loop, keep policy)	0.795 correct with 0.080 wrong — matches B's accuracy with 62% fewer errors
Highest precision regardless of coverage	Config D (loop, abstain policy)	0.070 wrong, but only 0.755 correct — dominated by Config C
No retrieval infrastructure at all	Config A is not viable	0.35 correct, 0.65 confidently wrong — never ship a closed-book QA system for factual queries

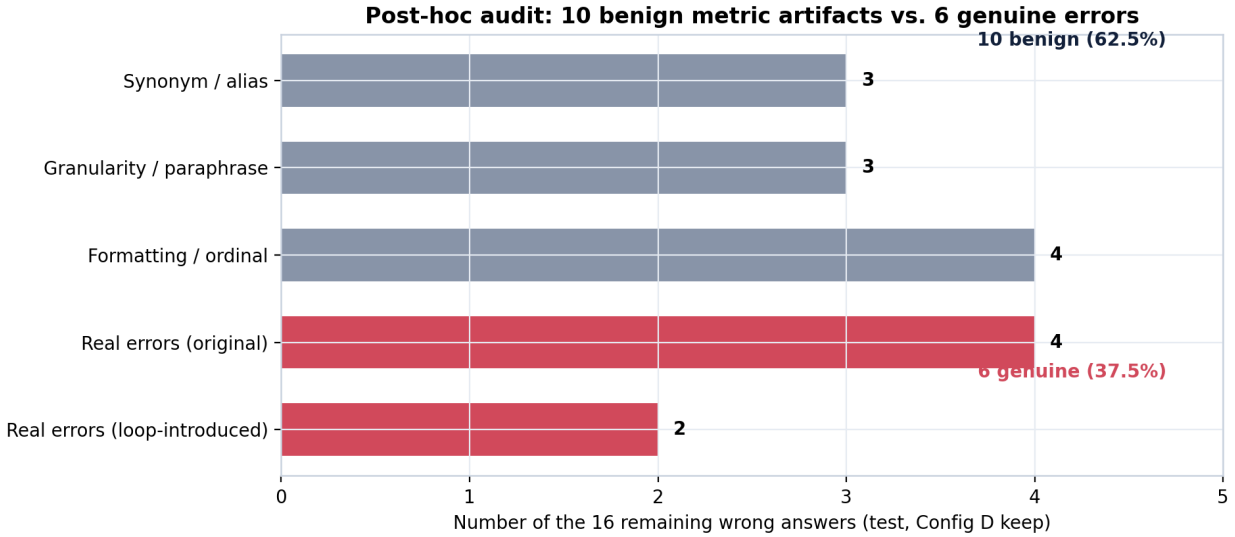
Cost of each option (test split)



- **Config C is free** relative to plain RAG (457 vs 446 tokens) and is the single best anti-hallucination intervention in the project.
- **Config D costs 1.8x Config C** but triggers on only ~22% of questions, so the marginal cost per question is much smaller than the worst case.
- **Retrieval itself costs ~15 ms** — negligible against a 0.8–1.0 s LLM call.

Post-Hoc Audit of the 16 Remaining Wrong Answers

Subjective, made after the test results were seen, and used to change no score



Reading	Count	Examples
Synonym / alias	3	"United States" for "American"; "Arctic Sea" for "Arctic Ocean"; "Torah" for "Law of Moses"
Granularity / paraphrase	3	"Washington, D.C." for "Capitol Hill"; "Director of Mad Max 2" for "Created by director George Miller"
Formatting / ordinal	4	"14th" for "14"; "Jim Diamond" for "James Aaron Diamond"
Real errors (original generation)	4	"Pilot" for "astronaut"
Real errors (introduced by the loop)	2	"Matt Maiellaro" for Dave Willis; "Las Vegas" for the Paradise casino

- **10 of 16 (62.5%) are benign metric artifacts** — the lenient metric does not recognise ordinals, nicknames, aliases or a different level of detail.
- **6 of 16 (37.5%) are genuine errors**: 4 from the original generation, 2 introduced by mitigation retries.
- **The 4 original-generation errors were never flagged** because the retrieved evidence supports their claims — no support-based detector can catch them.
- The audit covers only Configs C and D, so it **cannot be used to rank the configurations**.

11. Engineering Challenges

Eighteen problems hit during the project, and how each was solved

- Provider quota limits, reasoning starvation, FAISS segfaults
- Rate-limit violations, metric bias, NLI coreference breakdown
- Benchmark label noise, threshold uncertainty, fallback policy design

Engineering Challenges and Their Solutions (1 of 2)

The problems that actually cost us time

#	Challenge	Root cause	Engineering solution
1	API quota limits	Gemini free tier: 20 requests/day	Switched to Groq (`openai/gpt-oss-20b`); built a persistent SHA-256 disk cache
2	Reasoning token starvation	Thinking tokens consumed the whole `max_tokens` allocation	Lowered reasoning effort to `low`; auto-retry that doubles `max_tokens` up to 2048
3	Apple Silicon FAISS segfault	FAISS + PyTorch + Python 3.14 on macOS ARM64	Vectorized exact cosine with NumPy (`q @ emb.T`) — identical results in milliseconds
4	Rate-limit violations (HTTP 429)	Groq ceiling of 8,000 tokens/minute	Dynamic sliding-window limiter (2.2 s gap + 6,500 TPM budget) with exponential backoff
5	Evaluation metric bias	Strict string matching marked valid variations as wrong	Multi-tier metrics with lenient normalization and a manual short-form review
6	NLI coreference breakdown	Passages omit entity names or use pronouns	Novel Lexical Support metric — false alarms cut by more than 50%
7	Benchmark label noise	Some HaluEval 'hallucinated' answers are actually true	Diagnostic audit (`detector_analysis.py`) identifying mislabeled ground truth

Engineering Challenges and Their Solutions (2 of 2)

Methodological and reliability problems

#	Challenge	Root cause	Engineering solution
8	Threshold tuning uncertainty	Only 29 wrong answers on validation; 12 were short-form artifacts	Selection rule fixed in advance (F0.5 on pooled answers); stability confirmed by a sensitivity check
9	Mitigation fallback policy	Naive rejection of flagged answers discards valid answers	"Keep best-supported" policy — recovered 19 answers over 400 questions with 0 regressions
10	Grounded error masking	Grounded errors are supported by the evidence (synonyms, aliases)	Formulated E5/E6 experiments proving active mitigation beats passive rejection
11	Model availability (404)	`llama-3.1-8b-instant` not available on the account	Read the account's model list and chose `openai/gpt-oss-20b`
12	Local inference impossible	8 GB RAM; the laptop froze even on the smallest model	Hosted API for generation; embedding and NLI kept local on CPU
13	Free-tier token budget	~200K tokens/day for the whole project (~950K used)	Cache every call, smoke-test on 10 questions first, stage the heavy experiments over days
14	Misleading early log reading	Only 27 triggered questions on dev; round 2 looked marginal	Re-ran on validation unchanged + exact log analysis (`mitigation_analysis.py`)

The recurring theme: the hard problems were rarely the model. They were *measurement* problems — how to score an answer fairly, how to know whether a claim is supported, and how to avoid tuning on the thing you are trying to measure.

12. Limitations, Future Work & Reproducibility

What this study does not establish, and how to reproduce it exactly

- Sample size, metric and label limitations
- What the detector structurally cannot see
- Concrete next steps: stricter acceptance, alias-aware scoring, open-domain retrieval
- Full reproduction workflow and project structure

Limitations

Stated plainly — these bound every claim we make

- **Small samples.** Dev and validation are 100 questions each, test 200; all at temperature 0 with a single run. At n=100 a rate near 0.8 carries roughly ±8–9 points of margin, so small gaps (B vs C on accuracy) must not be read as real differences.
- **Detector results are uncertain.** With 200 balanced claims an AUC has a standard error of ~0.03; the chosen operating point rests on 24 flagged answers, so its precision and recall are uncertain by ~±0.2.
- **Scoring depends on gold-string matching.** Even the lenient and verified tiers compare against a single gold string, which penalizes synonyms and is generous toward hedged list answers.
- **The detector cannot see supported but non-responsive answers.** A "Yes." to a statement-style question, an incomplete answer, or a true statement that does not answer the question all produce a supported claim. These account for most of the wrong answers remaining after the loop.
- **Claim rewriting can lose the question's constraint** — a claim can be true and supported without answering the question.
- **Retrieval is easier than reality** — the corpus is the dataset's own knowledge field, not open-domain web search.
- **Reported latency is API call time only**; wall-clock time is larger because of deliberate rate limiting.
- **Prompts were never tuned** — one prompt per configuration throughout.

What a support-based detector CAN catch

- Ungrounded guesses made when retrieval failed (plain RAG)
- Claims contradicted by the retrieved evidence
- Answers introducing entities absent from the evidence

What it structurally CANNOT catch

- Supported but non-responsive answers ("Yes." to a statement-style question)
- Incomplete answers (a year instead of a full date)
- True statements that do not answer the question

We do not claim to eliminate hallucinations. We measure how much each stage improves reliability — and report the one pre-set criterion that was not met.

Future Work

Concrete, prioritised improvements — none of which were applied after seeing the test results

#	Improvement	Rationale (from our own error analysis)
1	Stricter acceptance rule for retried answers	The two wrong accepted retries had support 0.50 and 0.57 — barely above $\tau = 0.55$. A stricter gate (e.g. $\tau = 0.65$ for retries, or requiring agreement across both rounds) would reject both.
2	Check that the claim actually answers the question	The dominant residual error class is a supported claim that does not respond to the question ("Yes." to a statement-style question, a hedged list, an incomplete answer).
3	Alias-aware scoring	Ordinals ("14th" vs "14"), nicknames ("Jim" vs "James") and aliases ("Arctic Sea" vs "Arctic Ocean") account for a large share of the residual 'errors'.
4	A stronger verifier	The lexical score is a proxy for coreference. A coreference-aware or generation-based verifier (e.g. a QA model that must answer from the passage alone) should beat it.
5	Open-domain retrieval	The corpus is the dataset's own knowledge field; real deployment needs web-scale retrieval with a reranker.
6	Evaluate the remaining 300 test questions	Only the first 200 of 500 were used, once. Doubling the sample halves the confidence intervals.
7	Ablations E2/E3 (K and chunk size)	Top-K $\in \{1,3,5\}$ and chunk sizes $\{25,40,60\}$ words, run on dev/validation only.
8	Streamlit demo + publication figures	Interactive dashboard for live QA, evidence inspection, claim decomposition and retry visualisation.

Reproducibility

Every result in this deck can be regenerated from the repository

Complete reproduction workflow

```
# Stage 1 - data preparation & retrieval index
python src/make_splits.py            # download, clean, split (seed 42)
python src/build_index.py 120        # 10,040 chunks + 384-d embeddings
python src/eval_retrieval.py dev 120 # Recall@K, AnswerInChunk@K, MRR

# Stage 2 - baseline & RAG
python src/run_baseline.py dev
python src/run_rag.py dev b 3        # Config B
python src/run_rag.py dev c 3        # Config C
python src/rescore.py dev

# Stage 3 - claim extraction & verification
python src/make_claims.py dev
python src/verify.py dev
python src/detect_eval.py dev 0.5
python src/detector_analysis.py

# Stage 4 - validation & threshold tuning
python src/run_rag.py val b 3 && python src/run_rag.py val c 3
python src/make_claims.py val && python src/verify.py val
python src/tune_threshold.py val     # -> results/chosen_detector.json
python src/check_detector.py val     # sensitivity check

# Stage 5 - final evaluation
python src/run_baseline.py test 200
python src/run_rag.py test b 3 200 && python src/run_rag.py test c 3 200
python src/mitigate.py test 200      # Config D
python src/final_eval.py test        # bootstrap CIs + pre-set criteria
python src/verify_only.py dev val test   # E5 & E6
```

Determinism & artefacts

- **Fixed splits** (seed 42) with permanent row ids; temperature 0 for every LLM call.
- **Every LLM response cached on disk** (keyed by model, reasoning, temperature, token limit, prompt) with the model version logged alongside each answer.
- **All analysis re-uses saved predictions** — re-scoring, detector tuning, variant comparison and the mitigation analysis need **no new API calls**.
- **Frozen artefacts**: `results/chosen_detector.json` (variant, threshold, rule) and `data/mitigate_{dev,val,test}.jsonl` (full loop traces).

Stage	Token usage
Dev (baseline + B + C + claims)	~199K
Validation (B + C + claims)	~179K
Mitigation loop (dev + val)	~74K
Test (200 questions, all stages)	~500K
Total	**~950K**

Local compute (CPU only): embedding 10,040 chunks ≈ 20 s · retrieval ≈ 0.015 s/query · NLI on 765 test claims ≈ 613 s

13. Conclusions

What we set out to do, and what we actually established

- Retrieval is the dominant lever: +43.5 points of accuracy
- Grounding is the best anti-hallucination intervention: −14.5 points of errors at no significant accuracy cost
- The detector generalizes as a classifier but is limited as a filter
- Active mitigation beats passive rejection by 8.5 points of accuracy

Conclusions

Four findings, one negative result, and a clear recommendation

Finding	Evidence	Strength
Retrieval is the dominant lever	+43.5 pts accuracy vs. closed-book (CI [+36.0, +51.5]); replicated on all three splits	Strong
Grounded prompting is the best anti-hallucination intervention	−14.5 pts wrong-and-answered (CI [−20.0, −9.5]) at a statistically insignificant −3.0 pt accuracy cost; held on all three splits	Strong
The hybrid detector generalizes but is a moderate filter	Pair AUC 0.771 / 0.779 / 0.766 across dev / val / test; precision 0.52 on plain RAG, 0.22 on grounded answers	Moderate
Active mitigation beats passive rejection	Config D 0.795 correct vs. 0.710 for "B + verify" (CI [+5.0, +12.5]); +19 net correct answers over 400 questions with 0 regressions	Moderate
NEGATIVE RESULT: the loop does not reduce errors	Primary criterion NOT met — 16 wrong answers on test vs. C's 14; pooled wrong-and-answered unchanged at 23	Reported as-is

Our recommendation

Ship **Config C** (grounded RAG with abstention) as the default: it is free, it is the single most effective intervention, and it cuts hallucination by two thirds. Add **Config D**'s loop when answering more questions matters more than a marginally lower error rate — it matches plain RAG's accuracy with 62% fewer errors. Treat the detector as a **routing signal**, not a gate: use it to trigger re-retrieval, never to silently discard an answer.

Thank you

Questions & discussion

Team

- Thrivikram · 23BCS100
- Kuladeep · 23BDS021
- Harshith · 23BDS058
- Sadan · 23BDS048

Chaitra · 23BDS016

Course

Large Language Models · IIIT Dharwad

Resources used

- Dataset: HaluEval (RUCAIBox) — 10,000 QA rows
- Generator: openai/gpt-oss-20b via Groq
- Embeddings: sentence-transformers/all-MiniLM-L6-v2
- Verifier: MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli

Artefacts

README.md · results/report.md · src/ (29 scripts) · results/chosen_detector.json

Reference: Li, Cheng, Zhao, Nie & Wen. "HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models." EMNLP 2023.

Built with Sentence-Transformers, Hugging Face Transformers, the Groq API and scikit-learn.